

Week5 Session - 2 Facilitation Guide

(Understanding Node.js Modules & File System)

INDEX

- I. Recap of the previous session
- II. Common JS & ES6 Modules (require vs. import)
- III. Built-in Modules (fs, path, os, etc.)
- IV. Reading & Writing Files using fs module

(2hr ILT)

I. Recap of the previous session

In the previous session, we explored the foundational concepts of Node.js, a powerful JavaScript runtime environment used for building scalable and high-performance server-side applications. We began by discussing **what Node.js is**, emphasizing its ability to execute JavaScript code outside the browser and its unique event-driven, non-blocking architecture.

We then examined the **features and benefits of Node.js**, highlighting its asynchronous nature, fast execution with the V8 engine, and its vast ecosystem of reusable libraries available through npm (Node Package Manager). We also compared **Node.js with JavaScript**, focusing on their differences in usage - JavaScript for browser-based development and Node.js for server-side development.

The session included a practical segment on **installing Node.js and npm**, guiding students through downloading and setting up the runtime environment. Finally, we created a **simple Node.js program** to print "Hello, Node.js!" to the console, demonstrating how to write and execute Node.js scripts. This foundational understanding sets the stage for more advanced topics in Node.js development.

In this session we are going to learn CommonJS and ES6 modules in Node.js. Also, we will learn some built in modules available in Node.js.

II. CommonJS and ES6 Modules in Node.js

Node.js supports two module systems: **CommonJS (CJS)** and **ES6 Modules (ESM)**. These systems define how you can organize, share, and reuse code in your projects. While both systems serve the same purpose, they differ in syntax, functionality, and implementation. Below is a detailed explanation of each system.

CommonJS Module

CommonJS is the default module system used in Node.js. It allows developers to organize and structure their code into reusable modules. Each file in Node.js is treated as a separate module, and the `require()` function is used to load other modules.

Key Features of CommonJS:

- **Synchronous Module Loading:**
 - Modules are loaded synchronously in the order they appear in the code.

- Suitable for server-side development where files are loaded locally and quickly.
- **Module Scope:**
 - Each file in CommonJS has its own scope.
 - Variables declared in one file are private and not accessible in other files unless explicitly exported.
- **Exporting and Importing Modules:**
 - **Exporting:** Done using module.exports or exports.
 - **Importing:** Done using require().
- **Single and Multiple Exports:**
 - A module can export a single value or multiple values.
- **Used for Node.js:**
 - CommonJS was designed primarily for server-side JavaScript, and Node.js adopted it as its default module system.

CommonJS Module Workflow

1. Exporting Modules

You can export values, functions, or objects using module.exports or exports.

Example: Exporting a Single Value

Filename : rectangle.js

```
//function to calculate area of rectangle

function area(length,breadth){

    return `Area of rectangle :${length * breadth}`;

}

//exporting the function

module.exports = area
```

Example: Exporting Multiple Values

Filename : numericcalculation.js

```
//function to calculate sum of two numbers

function addition(num1,num2){

    return `Sum of ${num1} and ${num2} is : ${num1 + num2}`;

}

//-----

//function to calculate difference of two number
```

```

function difference(num1,num2){
    return `Difference between ${num1} and ${num2} is : ${num1 - num2}`;
}

//-----

//function to calculate product of two number
function product(num1,num2){
    return `Product of ${num1} and ${num2} is : ${num1 * num2}`;
}

//-----

//exporting functions
module.exports = {addition,difference,product};

```

2. Importing Modules

Use the require() function to import modules.

Example: Importing a Single Value

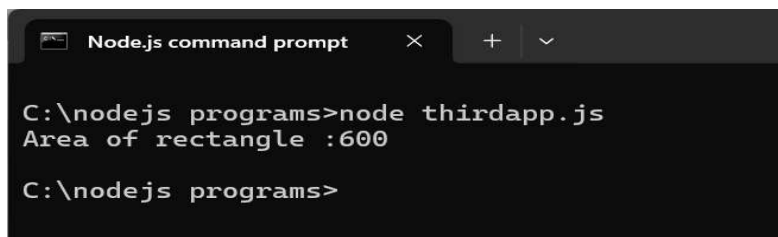
File: thirdapp.js

```

const area = require('./rectangle'); // Import rectangle.js
console.log(area(20,30)); // Output: 600

```

Output:



```

Node.js command prompt
C:\nodejs programs>node thirdapp.js
Area of rectangle :600
C:\nodejs programs>

```

Example: Importing Multiple Values

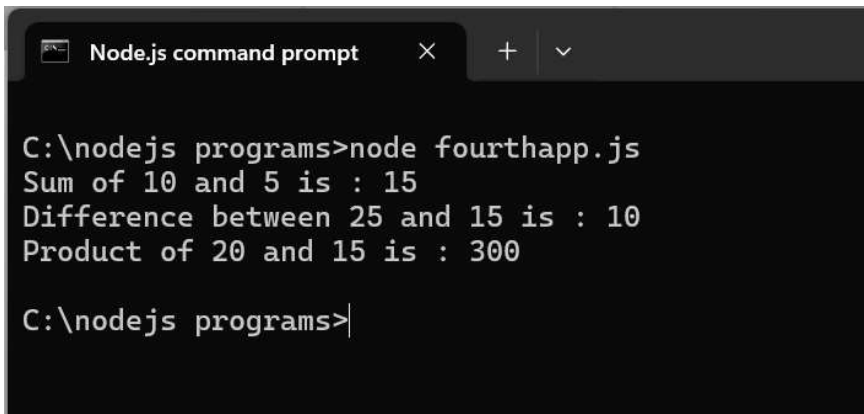
Filename : fourthapp.js

```

const calculation = require('./numericcalculation'); // Import numericcalculation.js
console.log(calculation.addition(10, 5));
console.log(calculation.difference(25, 15));
console.log(calculation.product(20, 15));

```

Output:-



```
Node.js command prompt X + v

C:\nodejs programs>node fourthapp.js
Sum of 10 and 5 is : 15
Difference between 25 and 15 is : 10
Product of 20 and 15 is : 300

C:\nodejs programs>|
```

Advantages of CommonJS

1. **Simple Syntax:** The require and module.exports syntax is straightforward.
2. **Wide Support:** It's supported by all Node.js versions by default.
3. **Synchronous Loading:** Ensures modules are fully loaded before use, reducing errors in server-side code.

Limitations of CommonJS

1. **Not Supported in Browsers:**
 - Browsers don't natively support CommonJS. You need tools like Webpack or Browserify to bundle CommonJS modules for browser use.
2. **No Tree-Shaking:**
 - Unused code cannot be automatically removed (unlike ES6 Modules).
3. **Synchronous Loading:**
 - Not ideal for large-scale, asynchronous environments (e.g., browsers).

Use Cases of CommonJS

1. Building server-side applications using Node.js.
2. Working with legacy Node.js codebases.
3. Simple projects without modern bundlers like Webpack.

Conclusion

- CommonJS (require and module.exports) is the default module system for Node.js.
- It's perfect for server-side applications where synchronous loading is not an issue.
- While it's gradually being replaced by ES6 Modules (import and export), it remains widely used due to its simplicity and legacy support.

ES6 Modules in Node.js

The ES6 module system (also called ECMAScript Modules or ESM) is the modern standard for JavaScript modules. It was introduced in ECMAScript 2015 (ES6) and provides a structured and powerful way to import and export functionality between files. Node.js started supporting ES6 modules natively in version 12 (with full support from version 14).

Key Features of ES6 Modules

1. **Static Import and Export:**
 - Modules are statically analyzed at compile time, enabling features like tree-shaking and faster execution.

- import is used to bring in modules, and export is used to share functionality between modules.
- 2. Asynchronous Loading:**
 - ES6 modules support asynchronous behavior and are suitable for both client and server environments.
 - 3. File-Based Modules:**
 - Each JavaScript file is treated as a module.
 - 4. Strict Mode by Default:**
 - ES6 modules run in strict mode ('use strict') by default, enhancing security and avoiding common mistakes.

How ES6 Modules Work

1. Exporting Modules

You can use the export keyword to make functions, objects, or variables available for other files.

Named Exports

You can export multiple values from a file using named exports.

File: math.js

```
//function to calculate sum of two numbers
export function addition(num1,num2){
    return `Sum of ${num1} and ${num2} is : ${num1 + num2}`;
}
//-----
//function to calculate difference of two number
export function difference(num1,num2){
    return `Difference between ${num1} and ${num2} is : ${num1 - num2}`;
}
//-----
//function to calculate product of two number
export function product(num1,num2){
    return `Product of ${num1} and ${num2} is : ${num1 * num2}`;
}
```

Default Exports

You can export a single default value from a module.

File: square.js

```
//function to calculate area of square
export default function area(side){
    return `Area of Square with Side ${side} is : ${side * side}`;
}
```

2. Importing Modules

You can use the import keyword to bring in exported functionality.

Importing Named Exports

You must use the same name as the exported variable or function.

File: fifthapp.js

```
import { addition, difference, product } from './math.js';
console.log(addition(10, 5));
console.log(difference(10, 5));
console.log(product(10, 5));
```

Output:

```
Node.js command prompt
C:\nodejs programs>node fifth.js
Sum of 10 and 5 is : 15
Difference between 10 and 5 is : 5
Product of 10 and 5 is : 50
C:\nodejs programs>
```

Importing Default Exports

You can use any name for the default export.

File: sixth.js

```
import area from './square.js';
console.log(area(40));
```

Output:

```
Node.js command prompt
C:\nodejs programs>node sixth.js
Area of Square with Side 40 is : 1600
C:\nodejs programs>
```

Using ES6 Modules in Node.js

To use ES6 modules in Node.js, follow these steps:

1. **File Extensions:**
 - Save the file with the .mjs extension, or set "type": "module" in your package.json.
2. **Setting "type" in package.json:**

```
{
  "type": "module"
}
```

3. **Run the Program:** Use the node command to execute the program:

```
node app.js
```

Comparison of ES6 Modules and CommonJS

Feature	ES6 Modules	CommonJS
Syntax	import / export	require / module.exports
Asynchronous Loading	Yes	No
Tree-Shaking Support	Yes	No

Default Usage in Node.js	No (needs "type": "module")	Yes
Usage in Browsers	Supported	Not supported natively
Scope	File-based	File-based

Advantages of ES6 Modules

- Static Analysis:**
 - Modules are statically analyzed at compile time, allowing for optimizations like tree-shaking.
- Interoperability:**
 - ES6 Modules are supported in modern browsers and can work on both the client and server.
- Cleaner Syntax:**
 - Provides a more intuitive and modern syntax for managing dependencies.
- Asynchronous Loading:**
 - Allows modules to load asynchronously, improving performance in large applications.

Conclusion

- ES6 modules (import / export) provide a modern, scalable way to manage dependencies in Node.js and browsers.
- Use them for better performance, cleaner syntax, and interoperability across environments.
- Ensure compatibility with Node.js by using .mjs files or setting "type": "module" in package.json.

III. Built-in modules of Node.js

Node.js comes with a variety of **built-in modules** that provide functionalities essential for building applications. These modules are part of Node.js itself, so you can use them without installing any external libraries. Below is a detailed explanation of some of the most commonly used built-in modules, their features, and examples.

1. fs (File System Module)

The fs module provides an API to interact with the file system, allowing you to read, write, update, delete, and perform other file operations.

Features:

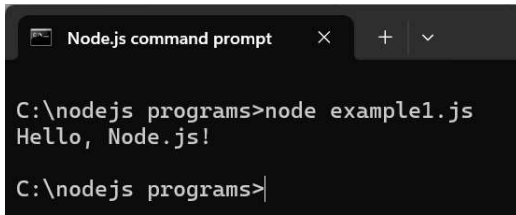
- Synchronous and asynchronous methods for file operations.
- Read/write files, directories, or metadata.
- File streams for large files.

Example:

```
const fs = require('fs');
// Writing to a file
fs.writeFileSync('example.txt', 'Hello, Node.js!'); // Synchronous
// Reading a file
fs.readFile('example.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log(data); // Output: Hello, Node.js!
```

```
});
```

Output:

A screenshot of a Node.js command prompt window. The title bar says "Node.js command prompt". The command prompt shows the directory "C:\nodejs programs" and the command "node example1.js" being executed. The output is "Hello, Node.js!".

```
C:\nodejs programs>node example1.js
Hello, Node.js!

C:\nodejs programs>
```

2. http (HTTP Module)

The http module allows you to create a web server and handle HTTP requests and responses.

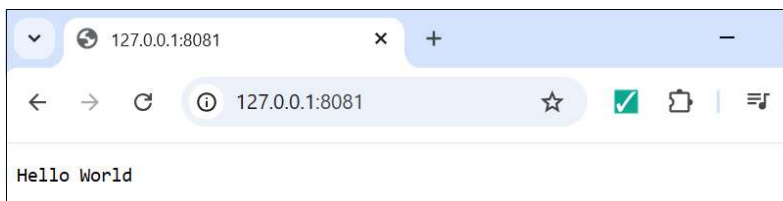
Features:

- Create servers and listen to requests.
- Handle different HTTP methods (GET, POST, etc.).
- Built-in support for headers and status codes.

Example:

```
var http = require("http");
http.createServer(function (request, response) {
  // Send the HTTP header
  // HTTP Status: 200 : OK
  // Content Type: text/plain
  response.writeHead(200, {'Content-Type': 'text/plain'});
  // Send the response body as "Hello World"
  response.end('Hello World\n');
}).listen(8081);
// Console will print the message
console.log('Server running at http://127.0.0.1:8081/');
```

Output:



3. url (URL Module)

The url module provides utilities for parsing and formatting URLs.

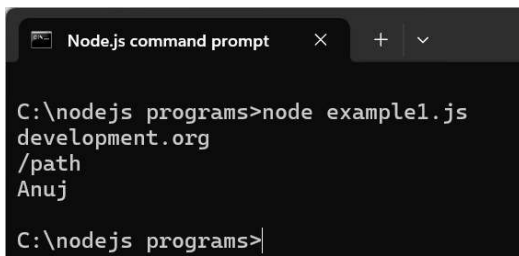
Features:

- Parse URLs into parts (protocol, hostname, path, query, etc.).
- Format a URL object into a URL string.

Example:

```
const url = require('url');
const myUrl = new URL('https://development.org:8080/path?name=Anuj&age=30');
// Access parts of the URL
console.log(myUrl.hostname);
console.log(myUrl.pathname);
console.log(myUrl.searchParams.get('name'));
```

Output:



```
Node.js command prompt
C:\nodejs programs>node example1.js
development.org
/path
Anuj
C:\nodejs programs>
```

4. path (Path Module)

The path module provides utilities to work with file and directory paths.

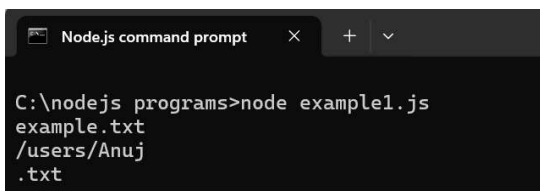
Features:

- Resolve, normalize, join, or parse file paths.
- Cross-platform compatibility for file paths.

Example:

```
const path = require('path');
console.log(path.basename('/users/Anuj/example.txt'));
console.log(path.dirname('/users/Anuj/example.txt'));
console.log(path.extname('/users/Anuj/example.txt'));
```

Output:



```
Node.js command prompt
C:\nodejs programs>node example1.js
example.txt
/users/Anuj
.txt
```

5. os (Operating System Module)

The os module provides operating system-related utility methods and properties.

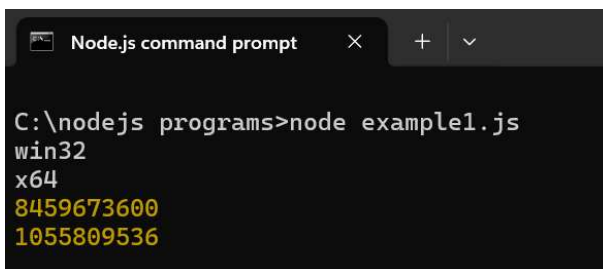
Features:

- Get system information like CPU, memory, and platform.
- Handle OS-specific features.

Example:

```
const os = require('os');
console.log(os.platform()); // Output: win32, linux, darwin, etc.
console.log(os.arch()); // Output: x64, arm, etc.
console.log(os.totalmem()); // Output: Total system memory in bytes
console.log(os.freemem()); // Output: Free memory in bytes
```

Output :



```
Node.js command prompt
C:\nodejs programs>node example1.js
win32
x64
8459673600
1055809536
```

6. events (Events Module)

The events module allows you to create, emit, and listen to custom events.

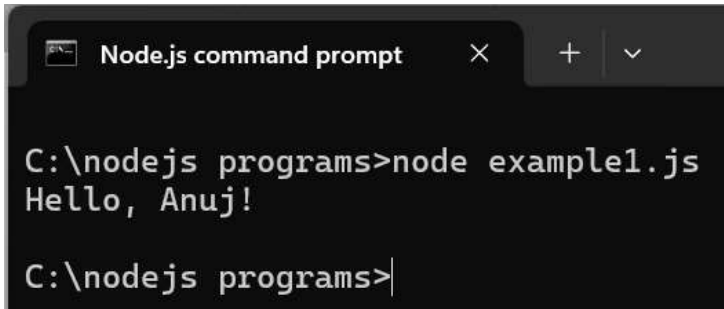
Features:

- Create and handle custom events.
- Central for building event-driven applications.

Example:

```
const EventEmitter = require('events');
const emitter = new EventEmitter();
emitter.on('greet', (name) => {
  console.log(`Hello, ${name}!`);
});
emitter.emit('greet', 'Anuj'); // Output: Hello, Anuj!
```

Output:



```
Node.js command prompt
C:\nodejs programs>node example1.js
Hello, Anuj!
C:\nodejs programs>
```

7. stream (Stream Module)

The stream module allows you to handle streaming data, such as reading or writing large files efficiently.

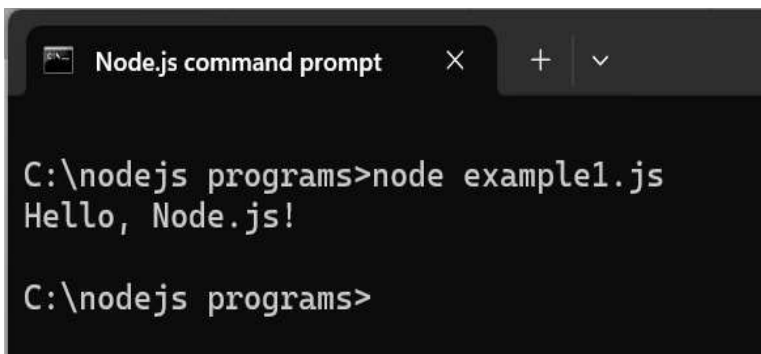
Features:

- Handle I/O in chunks (reading or writing).
- Provides readable, writable, and duplex streams.

Example:

```
const fs = require('fs');
// Reading a file as a stream
const readStream = fs.createReadStream('example.txt', 'utf8');
readStream.on('data', (chunk) => {
  console.log(chunk);
});
```

Output:



```
Node.js command prompt
C:\nodejs programs>node example1.js
Hello, Node.js!
C:\nodejs programs>
```

8. crypto (Cryptography Module)

The crypto module provides cryptographic functionality, including hashing, encryption, and decryption.

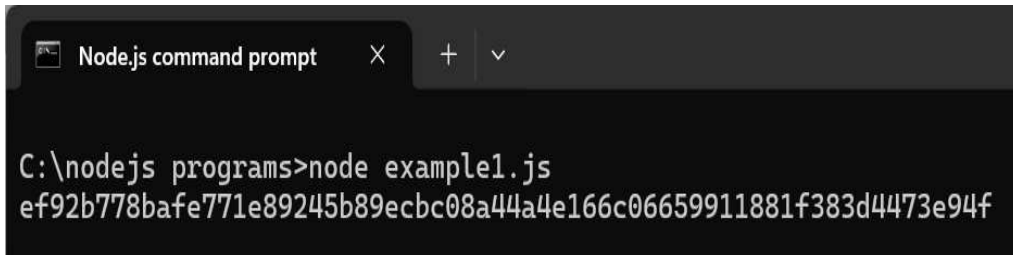
Features:

- Generate secure random data.
- Hashing (MD5, SHA256).
- Symmetric and asymmetric encryption.

Example:

```
const crypto = require('crypto');
// Hashing a string
const hash = crypto.createHash('sha256').update('password123').digest('hex');
console.log(hash); // Output: a hashed value
```

Output:



```
Node.js command prompt
C:\nodejs programs>node example1.js
ef92b778bafef771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f
```

9. util (Utilities Module)

The util module provides utility functions for debugging and working with Node.js features.

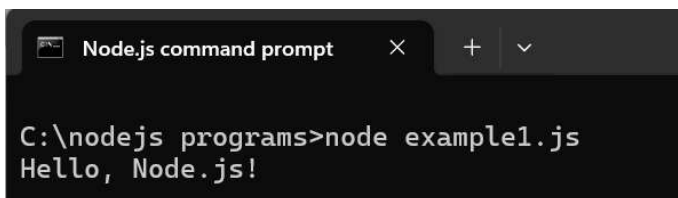
Features:

- Formatting strings.
- Inheriting prototypes.
- Promisifying callback-based functions.

Example:

```
const util = require('util');
const fs = require('fs');
const readFilePromise = util.promisify(fs.readFile);
readFilePromise('example.txt', 'utf8')
  .then((data) => console.log(data))
  .catch((err) => console.error(err));
```

Output:



```
Node.js command prompt
C:\nodejs programs>node example1.js
Hello, Node.js!
```

10. querystring (Query String Module)

The querystring module helps parse and format query strings.

Features:

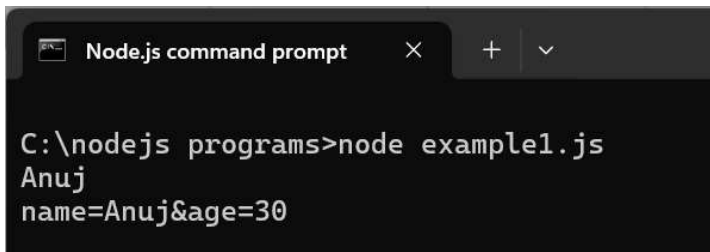
- Parse query strings into objects.
- Serialize objects into query strings.

Example:

```
const querystring = require('querystring');
const query = 'name=Anuj&age=30';
```

```
const parsedQuery = querystring.parse(query);
console.log(parsedQuery.name); // Output: Anuj
console.log(querystring.stringify(parsedQuery)); // Output: name=Anuj&age=30
```

Output:

A screenshot of a Node.js command prompt window. The title bar says "Node.js command prompt". The command prompt shows the command "C:\nodejs programs>node example1.js" and the output "Anuj" followed by "name=Anuj&age=30" on the next line.

```
C:\nodejs programs>node example1.js
Anuj
name=Anuj&age=30
```

11. timers (Timers Module)

The timers module provides functions to schedule code execution after a specified time.

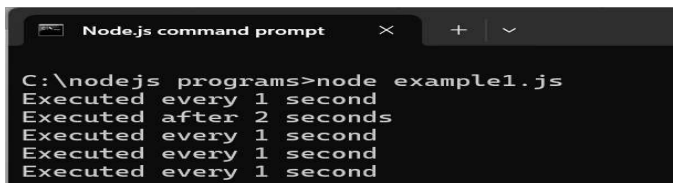
Features:

- Schedule recurring or one-time callbacks.

Example:

```
setTimeout(() => {
  console.log('Executed after 2 seconds');
}, 2000);
const interval = setInterval(() => {
  console.log('Executed every 1 second');
}, 1000);
setTimeout(() => clearInterval(interval), 5000); // Stops after 5 seconds
```

Output:

A screenshot of a Node.js command prompt window. The title bar says "Node.js command prompt". The command prompt shows the command "C:\nodejs programs>node example1.js" and the output "Executed every 1 second" repeated five times, with "Executed after 2 seconds" appearing in the middle of the sequence.

```
C:\nodejs programs>node example1.js
Executed every 1 second
Executed after 2 seconds
Executed every 1 second
Executed every 1 second
Executed every 1 second
```

12. child_process (Child Processes Module)

The child_process module allows you to spawn and manage external processes.

Features:

- Execute system commands.
- Fork processes for scalability.

13. zlib (Compression Module)

The zlib module provides compression and decompression utilities using the gzip and deflate algorithms.

These built-in modules in Node.js provide all the tools necessary for building a wide variety of applications, from file manipulation and web servers to cryptography and child processes. By combining these modules, you can build highly efficient and scalable applications.

IV. Reading & Writing Files using fs module

The fs (File System) module in Node.js allows you to perform file operations, such as reading from or writing to files. It provides both synchronous and asynchronous methods, giving you flexibility depending on the application's requirements.

1. Importing the fs module

To use the fs module, you must import it:

2. Writing Files

Asynchronous Writing: **fs.writeFile()**

This method writes content to a file asynchronously.

- If the file doesn't exist, it creates it.
- if it exists, it overwrites the content.

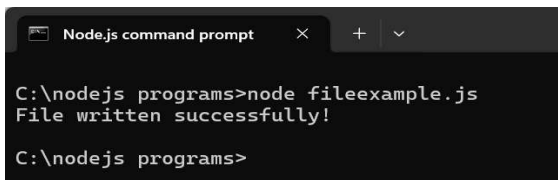
Syntax:

fs.writeFile(filePath, data, options, callback);

Example:

```
const fs = require('fs');
fs.writeFile('example.txt', 'Hello, World!', (err) => {
  if (err) {
    console.error('Error writing file:', err);
  } else {
    console.log('File written successfully!');
  }
});
```

Output:-

A screenshot of a Node.js command prompt window. The title bar says "Node.js command prompt". The command prompt shows the directory "C:\nodejs programs" and the command "node fileexample.js". The output is "File written successfully!".

```
Node.js command prompt
C:\nodejs programs>node fileexample.js
File written successfully!
C:\nodejs programs>
```

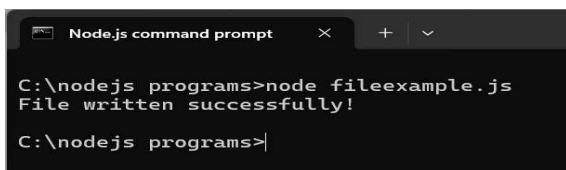
Synchronous Writing: **fs.writeFileSync()**

This method writes content to a file synchronously, blocking the event loop until the operation is complete.

Example:

```
const fs = require('fs');
try {
  fs.writeFileSync('example.txt', 'Hello, World!');
  console.log('File written successfully!');
} catch (err) {
  console.error('Error writing file:', err);
}
```

Output:

A screenshot of a Node.js command prompt window. The title bar says "Node.js command prompt". The command prompt shows the directory "C:\nodejs programs" and the command "node fileexample.js". The output is "File written successfully!".

```
Node.js command prompt
C:\nodejs programs>node fileexample.js
File written successfully!
C:\nodejs programs>
```

3. Reading Files

Asynchronous Reading: `fs.readFile()`

This method reads the content of a file asynchronously. You need to specify the file encoding (e.g., 'utf8') to get the file content as a string.

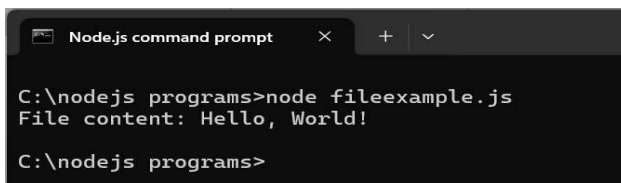
Syntax:

```
fs.readFile(filePath, encoding, callback);
```

Example:

```
const fs = require('fs');
fs.readFile('example.txt', 'utf8', (err, data) => {
  if (err) {
    console.error('Error reading file:', err);
  } else {
    console.log('File content:', data);
  }
});
```

Output:-

A screenshot of a Node.js command prompt window. The title bar says "Node.js command prompt". The command prompt shows the command `C:\nodejs programs>node fileexample.js` being executed. The output is `File content: Hello, World!`. The prompt then shows `C:\nodejs programs>` again.

```
Node.js command prompt
C:\nodejs programs>node fileexample.js
File content: Hello, World!
C:\nodejs programs>
```

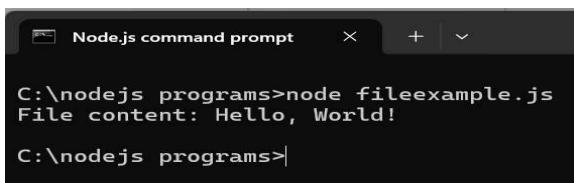
Synchronous Reading: `fs.readFileSync`

This method reads the file content synchronously and blocks the event loop until the operation completes.

Example:

```
const fs = require('fs');
try {
  const data = fs.readFileSync('example.txt', 'utf8');
  console.log('File content:', data);
} catch (err) {
  console.error('Error reading file:', err);
}
```

Output:-

A screenshot of a Node.js command prompt window. The title bar says "Node.js command prompt". The command prompt shows the command `C:\nodejs programs>node fileexample.js` being executed. The output is `File content: Hello, World!`. The prompt then shows `C:\nodejs programs>` again.

```
Node.js command prompt
C:\nodejs programs>node fileexample.js
File content: Hello, World!
C:\nodejs programs>
```

4. Appending to a File

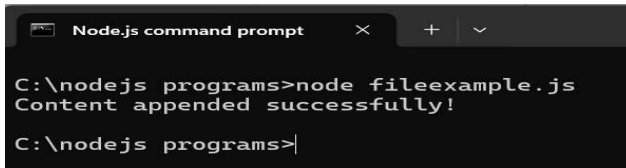
Asynchronous Appending: `fs.appendFile()`

This method adds new content to the end of a file asynchronously. If the file doesn't exist, it creates a new one.

Example:

```
const fs = require('fs');
fs.appendFile('example.txt', '\nAppended Content!', (err) => {
  if (err) {
    console.error('Error appending to file:', err);
  } else {
    console.log('Content appended successfully!');
  }
});
```

Output:-

A screenshot of a Node.js command prompt window. The title bar says "Node.js command prompt". The command prompt shows the directory "C:\nodejs programs" and the command "node fileexample.js" being executed. The output is "Content appended successfully!".

```
Node.js command prompt
C:\nodejs programs>node fileexample.js
Content appended successfully!
C:\nodejs programs>
```

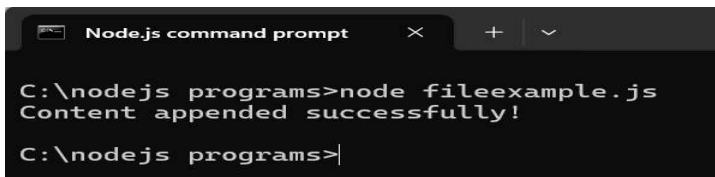
Synchronous Appending: fs.appendFileSync()

This method appends content to a file synchronously.

Example:

```
const fs = require('fs');
try {
  fs.appendFileSync('example.txt', '\nAppended Content!');
  console.log('Content appended successfully!');
} catch (err) {
  console.error('Error appending to file:', err);
}
```

Output:

A screenshot of a Node.js command prompt window. The title bar says "Node.js command prompt". The command prompt shows the directory "C:\nodejs programs" and the command "node fileexample.js" being executed. The output is "Content appended successfully!".

```
Node.js command prompt
C:\nodejs programs>node fileexample.js
Content appended successfully!
C:\nodejs programs>
```

5. Deleting Files

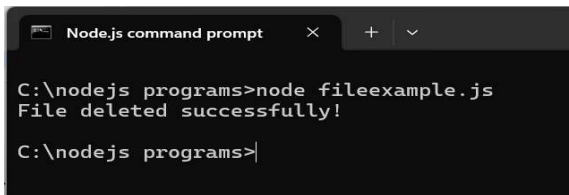
Asynchronous Deletion: fs.unlink()

This method deletes a file asynchronously.

Example:

```
const fs = require('fs');
fs.unlink('example.txt', (err) => {
  if (err) {
    console.error('Error deleting file:', err);
  } else {
    console.log('File deleted successfully!');
  }
});
```

Output:



```
Node.js command prompt
C:\nodejs programs>node fileexample.js
File deleted successfully!
C:\nodejs programs>
```

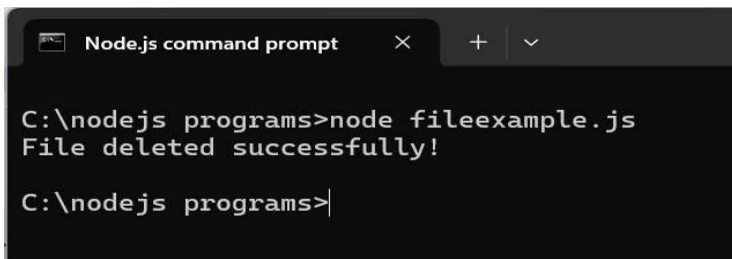
Synchronous Deletion: **fs.unlinkSync()**

This method deletes a file synchronously.

Example:

```
const fs = require('fs');
try {
  fs.unlinkSync('example.txt');
  console.log('File deleted successfully!');
} catch (err) {
  console.error('Error deleting file:', err);
}
```

Output:



```
Node.js command prompt
C:\nodejs programs>node fileexample.js
File deleted successfully!
C:\nodejs programs>
```

6. Checking File Existence

Deprecated: **fs.exists()**

The `fs.exists` method is deprecated; instead, you can use `fs.access`.

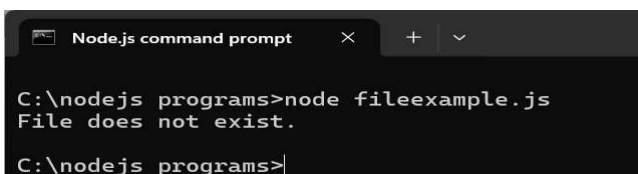
Modern Way: **fs.access()**

This method checks if a file exists and is accessible.

Example:

```
const fs = require('fs');
fs.access('example.txt', fs.constants.F_OK, (err) => {
  if (err) {
    console.log('File does not exist.');
```

Output:



```
Node.js command prompt
C:\nodejs programs>node fileexample.js
File does not exist.
C:\nodejs programs>
```

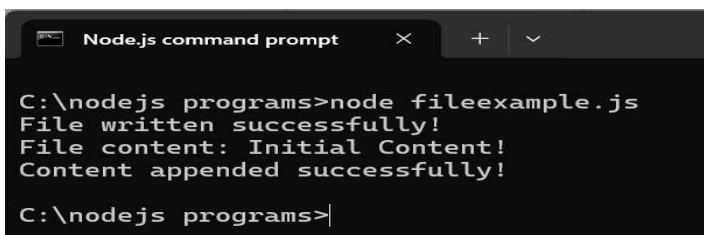

7. Practical Example: File Read and Write

Below is a simple program that writes content to a file, reads it back, and appends additional content:

```
const fs = require('fs');
// Step 1: Write content to a file
fs.writeFile('example.txt', 'Initial Content!', (err) => {
  if (err) {
    console.error('Error writing file:', err);
    return;
  }
  console.log('File written successfully!');

  // Step 2: Read the file content
  fs.readFile('example.txt', 'utf8', (err, data) => {
    if (err) {
      console.error('Error reading file:', err);
      return;
    }
    console.log('File content:', data);
    // Step 3: Append new content to the file
    fs.appendFile('example.txt', '\nNew Appended Content!', (err) => {
      if (err) {
        console.error('Error appending to file:', err);
      } else {
        console.log('Content appended successfully!');
      }
    });
  });
});
```

Output:

A screenshot of a Node.js command prompt window. The title bar says "Node.js command prompt". The command prompt shows the following sequence of commands and outputs:
C:\nodejs programs>node fileexample.js
File written successfully!
File content: Initial Content!
Content appended successfully!
C:\nodejs programs>

8. Real-World Use Cases

1. **Configuration Files:** Store and load application configurations.
2. **Log Files:** Write logs for debugging or monitoring purposes.
3. **Data Storage:** Store user data or temporary results.
4. **Template Files:** Read and render HTML templates in server-side applications.

These examples showcase how to efficiently use the fs module to handle file operations in Node.js.

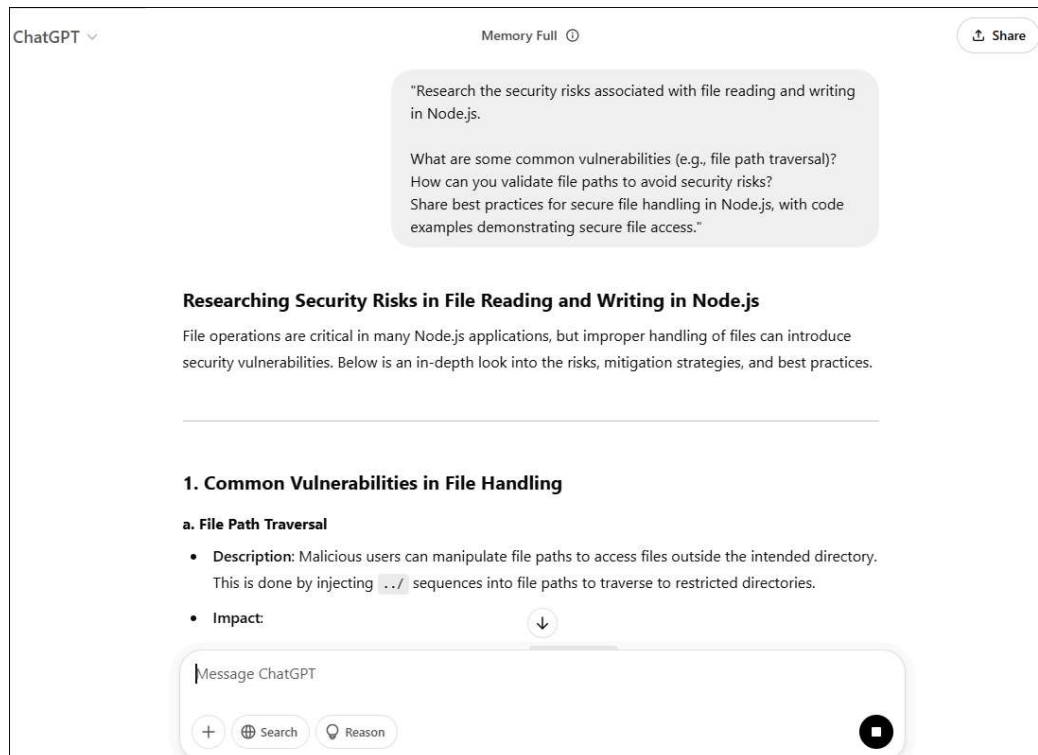
Exercise:

Use GPT and explore Security Considerations in File Operations.

Put the below problem statement in the message box and see what ChatGPT says.

“Research the security risks associated with file reading and writing in Node.js.

- What are some common vulnerabilities (e.g., file path traversal)?
- How can you validate file paths to avoid security risks?
- Share best practices for secure file handling in Node.js, with code examples demonstrating secure file access.”



After the 1.5 hr ILT, the student has to do a 30 minute live lab. Please refer to the Week5 Session- 2 Lab doc in the LMS.